

TÀI LIỆU KỸ THUẬT

LUỒNG XỬ LÝ TÍNH NĂNG REPORT

V2.1 — FINAL ARCHITECTURE

Snapshot Architecture · Integrity Metadata · Defense-in-Depth

Phiên bản	V2.1 — Final Architecture
Dựa trên	V2.0 (Snapshot Architecture) + Technical Audit
Thay đổi chính	Rate limit sliding window · Anchor Selection · Spot-check tamper detection · Integrity Metadata · localStorage key isolation
Stack	Spring Boot · Node.js · MongoDB · RabbitMQ · Socket.IO · Redis

PHẦN 1: TỔNG QUAN KIẾN TRÚC

V2.1 xây dựng trên nền tảng Client-Side Snapshot của V2.0, bổ sung ba lớp bảo vệ mới: Integrity Metadata chống thao túng ngữ cảnh, Spot-check Validation chống fabricated evidence, và Rate Limiting sliding window thay thế hard limit.

1.1 Nguyên tắc thiết kế cốt lõi

- Bằng chứng bất biến (Immutable Evidence): nội dung tin nhắn được niêm phong tại thời điểm tạo report, không phụ thuộc vào trạng thái server sau đó.
- Defense-in-Depth: không có lớp bảo vệ đơn lẻ nào là đủ — hệ thống kết hợp validation DB, spot-check service, metadata tín hiệu và UI warning.
- Minimal Disclosure: admin chỉ được truy cập đúng và đủ dữ liệu cần thiết để ra quyết định, không hơn.
- Pragmatic Security: các giải pháp bảo mật được cân bằng với scope và timeline của dự án thực tế.

1.2 Bảng so sánh V2.0 → V2.1

Hạng mục	Trạng thái V2.0	Thay đổi trong V2.1
Evidence integrity (revoke attack)	✓ Đã giải quyết bằng Snapshot	Giữ nguyên
Rate limiting	3 reports/ngày — quá chặt	Sliding window: 5/giờ · 20/ngày · 1/target/ngày
Deduplication (DB layer)	✓ Compound Unique Index	Giữ nguyên
Privacy — Deep Fetch	✓ Đã loại bỏ	Giữ nguyên
GROUP ban flow	✓ DISBAND_GROUP event	Giữ nguyên
Admin race condition	✓ IN_PROGRESS lock	Giữ nguyên
Feign JWT async	✓ Loại bỏ bởi Snapshot	Giữ nguyên
DLQ resilience	✓ Dead Letter Queue	Giữ nguyên
Customize Mode	✗ Bị xóa hoàn toàn	Thay bằng Anchor Selection (±15 quanh anchor)
Snapshot tampering	✗ Attack surface mới	Spot-check validation trong Service Layer
Context manipulation	✗ Không có cơ chế phát hiện	Anchor Selection (triệt tiêu gap) + Mini-Timeline (3 trường metadata)
Admin notification persistence	localStorage key chung	Key động: admin_noti_{adminId}

PHẦN 2: DATA MODEL & DATABASE DESIGN

2.1 Entity Report (MongoDB collection: reports)

Trường message_ids của V1 đã bị loại bỏ hoàn toàn từ V2.0. V2.1 mở rộng cấu trúc message_snapshots với Integrity Metadata.

Field	Mô tả
id (String / ObjectId)	PK tự sinh
reporter_id (String, indexed)	UUID của người gửi báo cáo
target_id (String, indexed)	UUID/ObjectId của đối tượng bị báo cáo
target_type (Enum)	USER hoặc GROUP
target_name (String)	Tên đối tượng — lưu tĩnh tại thời điểm tạo
reason (Enum)	Lý do báo cáo — xem Enum ReportReason
description (String, nullable)	Mô tả bổ sung — tùy chọn
message_snapshots (List<Object>)	Mảng tối đa 31 object — cấu trúc chi tiết tại mục 2.2
status (Enum)	PENDING · IN_PROGRESS · RESOLVED · REJECTED
admin_notes (String, nullable)	Ghi chú của admin khi xử lý
resolved_by (String, nullable)	Email / ID admin đã xử lý
resolved_action (Enum, nullable)	DISMISS · WARN · BAN · DISBAND_GROUP
created_at (LocalDateTime)	Tự động sinh bởi @CreateDate
image_urls (List<String>)	URL ảnh bằng chứng (Tùy chọn, đặc biệt quan trọng cho GROUP report hoặc khi tin nhắn vi phạm nằm ngoài vùng 31 tin snapshot)
updated_at (LocalDateTime)	Thời gian cập nhật trạng thái cuối cùng (Dùng để tracking thời gian lock IN_PROGRESS của Admin)

2.2 Cấu trúc MessageSnapshot Object (MỚI trong V2.1)

Mỗi phần tử trong mảng message_snapshots gồm các trường sau. Các trường Integrity Metadata được đánh dấu [IM] là bổ sung mới so với V2.0.

Trường	Mô tả
messageId (String)	ID gốc của tin nhắn — dùng cho spot-check
senderId (String)	UUID của người gửi tin nhắn
content (String)	Nội dung tin nhắn tại thời điểm tạo report
contentType (String)	TEXT · IMAGE · FILE · STICKER
sentAt (LocalDateTime)	Thời điểm gửi tin nhắn gốc
isAnchor (Boolean)	[IM] true nếu đây là Anchor Message được người dùng chọn
sequenceIndex (Integer)	[IM] Vị trí tuyệt đối của tin nhắn trong toàn bộ conversation
totalMessagesInConversation	[IM] Tổng số tin nhắn trong conversation lúc tạo report

(Integer)	
isByReporter (Boolean)	[IM] true nếu senderId == reporter_id — để tính tỉ lệ xuất hiện của reporter trong evidence

2.3 Database Constraints

- Compound Unique Index trên { reporter_id, target_id, reason } với filter điều kiện status: PENDING — ngăn tạo report trùng lặp ở tầng DB, tự động release khi report được xử lý xong.
- Text Index trên target_name — thay thế regex full scan, hỗ trợ \$text query hiệu quả cho admin search.
- TTL Index tùy chọn trên created_at cho REJECTED reports sau 90 ngày — giảm storage.

2.4 Enums

Enum	Giá trị
ReportReason	SCAM_FRAUD · SPAM_HARASSMENT · SEXUAL_CONTENT · VIOLENCE_TERRORISM · CHILD_ABUSE · OTHER
ReportStatus	PENDING · IN_PROGRESS · RESOLVED · REJECTED
TargetType	USER · GROUP
AdminAction	DISMISS · WARN · BAN · DISBAND_GROUP

PHẦN 3: RATE LIMITING — SLIDING WINDOW (V2.1)

Breaking change so với V2.0: Giới hạn 3 reports/ngày đã bị loại bỏ. Lý do: hard limit 3/ngày vô hiệu hóa khả năng report của nạn nhân trong các tình huống brigading (nhiều tài khoản spam đồng thời). Sliding window cho phép kiểm soát burst trong khi vẫn bảo vệ nạn nhân thực sự.

3.1 Kiến trúc ba lớp Redis counter

Ba Redis key độc lập được duy trì cho mỗi request tạo report:

Redis Key	TTL	Giới hạn & Mục đích
rate:report:{userId}:hour	3600 giây (1 giờ)	Max 5 — chống burst spam ngắn hạn
rate:report:{userId}:day	86400 giây (24 giờ)	Max 20 — giới hạn tổng ngày, bảo vệ queue admin
rate:report_target:{reporterId}:{targetId}:day	86400 giây (24 giờ)	Max 1 — ngăn spam report nhắm vào 1 người cụ thể (trả đũa cá nhân)

Key thứ ba (per-target) là 'viên đạn bạc' chống revenge reporting: user A report user B, bị B report lại liên tục vì tức giận. Key này bị reset khi report hiện tại được RESOLVED hoặc REJECTED.

3.2 Pseudo-code triển khai (API Gateway / ReportService)

```
// Kiểm tra theo thứ tự: per-target trước (rẻ nhất), sau đó per-hour, per-day
String targetKey = 'rate:report_target:' + reporterId + ':' + targetId + ':day';
String hourKey   = 'rate:report:' + reporterId + ':hour';
String dayKey    = 'rate:report:' + reporterId + ':day';

if (redis.incr(targetKey) > 1) throw new RateLimitException('DUPLICATE_TARGET');
if (redis.incr(hourKey)   > 5) throw new RateLimitException('HOURLY_LIMIT');
if (redis.incr(dayKey)    > 20) throw new RateLimitException('DAILY_LIMIT');

// Chỉ set TTL khi tạo key lần đầu (incr trả về 1)
// redis.expire(key, ttl) được gọi trong cùng pipeline
```

3.3 Response cho client

HTTP Status	Trường hợp
429 Too Many Requests	Vi phạm bất kỳ giới hạn nào
Header: Retry-After	Số giây còn lại của TTL tương ứng
Header: X-RateLimit-Reason	DUPLICATE_TARGET · HOURLY_LIMIT · DAILY_LIMIT

PHẦN 4: LƯỒNG UI/UX TẠO BÁO CÁO (Frontend → Backend)

4.1 Luồng 1 — Báo cáo tin nhắn cụ thể (Anchor Selection)

V2.1 thay thế *Customize Mode* (checkbox thủ công) bằng *Anchor Selection*. Điểm neo là tin nhắn user nhấn giữ, hệ thống tự động lấy ± 15 tin xung quanh — user kiểm soát tâm điểm, không kiểm soát từng tin lẻ.

- Điểm kích hoạt: User nhấn giữ (long-press) vào 1 tin nhắn cụ thể → chọn 'Báo cáo'.
- Phân nhánh Điều hướng (Routing):
 - Nếu là Chat 1-1: Hệ thống ngầm định `target_type = USER`, `target_id = friendId`.
 - Nếu là Group Chat: Modal hiển thị: "Bạn muốn báo cáo ai?"
 - Tùy chọn A (Thành viên gửi tin này): `target_type = USER`, `target_id = senderId`.
 - Tùy chọn B (Toàn bộ nhóm này): `target_type = GROUP`, `target_id = groupId`.
- Gom bằng chứng: Frontend gán tin nhắn được chọn là Anchor Message (`isAnchor = true`). Hàm `generateEvidenceSnapshot(anchorId, conversationMessages)` trích xuất tối đa 15 tin trước + Anchor + 15 tin sau (chuỗi 31 tin liên tiếp, không khoảng trống).
- Metadata: Với mỗi tin nhắn, frontend đính kèm `sequenceIndex`, `totalMessagesInConversation`, `isByReporter`.
- Frontend hiển thị preview 31 tin, tin `isAnchor = true` được highlight.
- User chọn Lý do (Reason), nhập Mô tả (tùy chọn), upload thêm ảnh chụp màn hình (`image_urls`) nếu cần, sau đó nhấn Gửi → `POST /api/v1/reports`.

4.2 Luồng 2 — Báo cáo từ Menu Hội thoại nhóm (No Anchor Selection)

- Điểm kích hoạt: User bấm nút 'Báo cáo' từ Menu Thông tin bạn bè (1-1) hoặc Menu Thông tin Nhóm (Group Info).
- Phân nhánh Điều hướng (Routing):
 - Nếu là Chat 1-1: Hệ thống ngầm định `target_type = USER`, `target_id = friendId`.
 - Nếu là Group Chat: Modal Interstitial hiển thị: "Bạn muốn báo cáo ai?"
 - Tùy chọn A (Thành viên cụ thể): Click vào tên `$\rightarrow$` `target_type = USER`, `target_id = memberId`.
 - Tùy chọn B (Báo cáo cả nhóm): Click nút dưới cùng `$\rightarrow$` `target_type = GROUP`, `target_id = groupId`.
- Gom bằng chứng: Không có Anchor trong luồng này `$\rightarrow$` hàm `generateEvidenceSnapshot()` tự động cắt lấy 30 tin nhắn mới nhất tính từ dưới lên (`isAnchor` luôn bằng `false`).
- Metadata: Tính toán đính kèm `sequenceIndex`, `totalMessagesInConversation`, `isByReporter` tương tự Luồng 1.

5. User chọn Lý do, nhập Mô tả, upload ảnh đính kèm (bắt buộc phải có ảnh nếu vi phạm nằm ở tít trên cao, ngoài phạm vi 30 tin mới nhất), sau đó nhấn Gửi → POST /api/v1/reports.

4.3 Validation tại API Gateway — POST /api/v1/reports

- Rate Limit Check: ba Redis counter theo mục 3.2.
- Snapshot Size: `message_snapshots.length <= 31`.
- Spot-check Tamper Detection: xem chi tiết tại Phần 5.
- Compound Unique Index: nếu DB throw `DuplicateKeyException` → HTTP 409 Conflict với message 'Báo cáo tương tự đang chờ xử lý'.

PHẦN 5: SPOT-CHECK TAMPER DETECTION (MỚI TRONG V2.1)

Client-Side Snapshot tạo ra attack surface: bad actor có thể intercept request và sửa nội dung trong message_snapshots trước khi gửi lên server, fabricating evidence. Spot-check giải quyết vấn đề này với chi phí chỉ 1 query DB.

5.1 Nguyên lý hoạt động

- Server chọn ngẫu nhiên 1 MessageSnapshot từ mảng đầu vào.
- Gọi messageService.getByld(randomSample.messageId) để lấy bản gốc từ message-service.
- Nếu bản gốc tồn tại (tin chưa bị revoke) và content không khớp → throw TamperedEvidenceException.
- Nếu bản gốc là null (tin đã bị revoke) → snapshot hợp lệ, đây là tính năng core của kiến trúc.

5.2 Pseudo-code — Service Layer (ReportService.java)

```
private void spotCheckSnapshot(List<MessageSnapshot> snapshots) {
    // Lọc ra các tin chưa bị revoke (có thể verify)
    List<MessageSnapshot> verifiable = snapshots.stream()
        .filter(s -> !s.isRevokedAtReport())
        .collect(Collectors.toList());

    if (verifiable.isEmpty()) return; // Tất cả đã revoke — không thể verify

    // Chọn ngẫu nhiên 1 tin để spot-check
    MessageSnapshot sample = verifiable.get(
        ThreadLocalRandom.current().nextInt(verifiable.size()));

    Message actual = messageServiceClient.getByld(sample.getMessageId());

    if (actual != null &&
        !actual.getContent().equals(sample.getContent())) {
        log.warn('Tampered snapshot detected. reporterId={}, messageId={},
            sample.getMessageId());
        throw new TamperedEvidenceException(
            'Snapshot content does not match server record');
    }
}
```

5.3 Trade-off và giới hạn

Spot-check 1 tin cho xác suất phát hiện là $1/N$ (N = số tin trong snapshot). Với $N = 31$, xác suất phát hiện 1 lần ~ 3.2%. Với 5 lần report liên tiếp ~ 15%. Đây là deterrence, không phải

guarantee — nhưng với cost chỉ 1 DB query, đây là trade-off tốt nhất cho scope đề án. Bad actor không biết tin nào sẽ bị check, nên việc chỉnh sửa nhiều tin không an toàn hơn.

PHẦN 6: LUỒNG QUẢN TRỊ VIÊN (Admin Dashboard)

6.1 Admin Review Modal — ReportActionModal

1. Concurrency Lock: khi admin mở modal, PATCH /reports/{id}/lock được gọi → status chuyển IN_PROGRESS, lockedBy = adminId, lockedAt = now(). Nếu report đã IN_PROGRESS bởi admin khác → hiển thị warning 'Đang được xử lý bởi [admin]'.
2. Auto-release: Frontend gửi heartbeat PATCH /reports/{id}/heartbeat mỗi 2 phút khi modal đang mở. Nếu lock quá 30 phút không có heartbeat → backend tự reset về PENDING.
3. Hiển thị Evidence: render trực tiếp từ message_snapshots trong MongoDB — không có bất kỳ fetch động nào. Admin thấy đúng những gì reporter đã niêm phong.
4. Mini Timeline: hiển thị vị trí evidence trong tổng thể conversation dựa trên sequenceIndex và totalMessagesInConversation.

6.2 Hiển thị ngữ cảnh Timeline (Mini-Timeline)

Thay vì chấm điểm rườm rà, Admin Dashboard sử dụng sequenceIndex và totalMessagesInConversation để vẽ một thanh Progress Bar nhỏ gọn. Thanh này cho Admin biết bằng chứng 31 tin nhắn đang nằm ở phần Mở đầu, Khúc giữa, hay Kết thúc của cuộc hội thoại, giúp định vị bối cảnh một cách trực quan.

6.3 Admin Actions

Hành động	Kết quả & Event phát ra
DISMISS	Report → REJECTED. Không phát event. Dừng khi bằng chứng mơ hồ hoặc hai bên đùa giỡn.
WARN	Report → RESOLVED. Phát user.warned → user-service tăng warningCount += 1. Nếu warningCount >= 3 → tự động BAN.
BAN	Report → RESOLVED. Phát user.banned → user-service set status = BANNED → realtime-service force disconnect socket.
DISBAND_GROUP	Report → RESOLVED. Phát group.disbanded → group-service xóa/disable group → realtime-service kick toàn bộ thành viên.

PHẦN 7: XỬ LÝ BẤT ĐỒNG BỘ, 3-STRIKE & DLQ

7.1 Quy tắc 3-Strike

- Admin chọn WARN → report-service publish user.warned event.
- user-service consume event → warningCount += 1.
- Nếu warningCount >= 3 → tự động set status = BANNED, publish user.banned event tiếp theo.
- realtime-service consume user.banned → force disconnect socket của user bị ban.

7.2 RabbitMQ Exchange & Routing Keys

Routing Key	Producer → Consumer
user.warned	report-service → user-service
user.banned	report-service / user-service (3-strike) → user-service · realtime-service
group.disbanded	report-service → group-service · realtime-service
report.auto_resolved	report-service (auto-resolve) → realtime-service (notify reporter)

7.3 Dead Letter Queue (DLQ) — Bảo toàn án phạt

- Áp dụng DLQ cho tất cả queue nhạy cảm: user.warned.queue, user.banned.queue, group.disbanded.queue.
- Cấu hình x-dead-letter-exchange và x-message-ttl trên mỗi queue.
- Nếu consumer throw exception hoặc service down, message được chuyển sang DLQ thay vì bị drop.
- Án phạt được thực thi ngay khi service phục hồi và consume lại từ DLQ.
- DLQ monitoring: admin có thể xem số lượng message trong DLQ để phát hiện service failure sớm.

7.4 Auto-resolve Duplicates — Phát Event

V2.1 đảm bảo mọi lần chuyển status sang RESOLVED đều publish event report.auto_resolved, kể cả khi được resolve tự động bởi SYSTEM (sau khi admin BAN một user, toàn bộ PENDING report về user đó tự động RESOLVED).

- Reporter nhận notification qua Socket.IO khi report của mình được resolve.
- resolvedBy = 'SYSTEM' trong trường hợp auto-resolve — reporter thấy 'Báo cáo đã được xử lý'.

PHẦN 8: ADMIN NOTIFICATION PERSISTENCE

V2.0 dùng localStorage với key chung — lỗi bảo mật: admin A và admin B dùng chung máy sẽ thấy notification của nhau. V2.1 sửa bằng key động.

8.1 Giải pháp — Zustand Persist với key động

Frontend sử dụng Zustand middleware persist với cấu hình key phụ thuộc vào adminId:

```
// store/notificationStore.ts
const useNotificationStore = create(
  persist(
    (set, get) => ({
      notifications: [],
      addNotification: (n) => set(state => ({
        notifications: [n, ...state.notifications].slice(0, 100)
      })),
      clearAll: () => set({ notifications: [] }),
    }),
    {
      name: 'admin_noti_' + getCurrentAdminId(), // Key động
      partialize: (state) => ({ notifications: state.notifications }),
    }
  )
);
```

getCurrentAdminId() lấy ID từ JWT đã decode trong auth store — không cần request thêm. Mỗi admin có namespace localStorage riêng biệt.

8.2 Giới hạn đã biết

- Vẫn là client-side storage: mất data khi xóa cache trình duyệt — chấp nhận được cho scope đồ án.
- Không đồng bộ giữa nhiều thiết bị của cùng một admin — giải pháp đầy đủ cần collection admin_notifications trong MongoDB, nằm ngoài scope V2.1.

PHẦN 9: BẢNG GIẢI QUYẾT TECHNICAL DEBT TOÀN DIỆN

V2.1 chính thức giải quyết tất cả vấn đề từ audit V1 và V2.0:

Vấn đề gốc (V1)	Trạng thái V2.1	Giải pháp áp dụng
IN_PROGRESS không được dừng	✓ Resolved	API lock khi admin mở modal + heartbeat auto-release
Không có rate-limiting	✓ Resolved	Redis sliding window: 5/giờ · 20/ngày · 1/target/ngày
Không deduplication phía tạo	✓ Resolved	MongoDB Compound Unique Index (reporter, target, reason) filter PENDING
FeignConfig relay JWT async	✓ Resolved	Eliminated: Snapshot đóng gói đủ data, không cần Feign async
resolvedAction DTO coupling	✓ Resolved	Entity Report độc lập hoàn toàn với DTO, ánh xạ qua Service Layer
GROUP report không có BAN logic	✓ Resolved	AdminAction.DISBAND_GROUP + event group.disbanded chuyên biệt
Auto-resolve không publish event	✓ Resolved	Mọi status → RESOLVED đều đi qua gateway publish RabbitMQ
Notification store in-memory	✓ Resolved	Zustand persist với key động admin_noti_{adminId}
Evidence bị revoke trước review	✓ Resolved	Client-Side Snapshot: nội dung niêm phong tại thời điểm tạo report
Privacy: Deep Fetch 200 messages	✓ Resolved	Loại bỏ hoàn toàn fetchFullConversationHistory
Context manipulation / cherry-picking	✓ Resolved	Integrity Metadata (3 trường)
Snapshot tampering (V2.0 attack surface)	✓ Resolved	Spot-check: verify ngẫu nhiên 1 tin với message-service
Rate limit quá chặt (V2.0)	✓ Resolved	Sliding window thay thế hard limit 3/ngày
Customize Mode bị xóa (V2.0)	✓ Resolved	Anchor Selection: user chọn tâm điểm, hệ thống lấy ±15 tự động
localStorage key chung (V2.0)	✓ Resolved	Key động phụ thuộc adminId